IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

P11 – Initiation à la programmation

-

Introduction et notions de base

Hervé L'helguen

Préambule

P11 : de septembre à janvier

7 CM de 2 heures (Hervé L'helguen)

10 TD de 2 heures

- Groupe TD12 : M. Antoine Bralet
- Groupe TD34 : Mme Aurélie Leborgne (puis M. Ilias Rmouque après novembre)
- Groupe TD56 : M. Antoine Bralet
- Groupe TD78 : M. Hervé L'helguen

27 TP de 2 heures, idem sauf :

- groupe TP1 et TP2 : Mme Murielle Torregrossa
- groupe TP5 et TP6 : Mme Véronique Richard
- groupe TP3 et TP4 : M. Ilias Rmouque (après novembre)

Préambule

Les évaluations

- 3 évaluations papier lors des TD (20 min)
- 1 évaluation intermédiaire après la Toussaint (1 heure)
- 1 évaluation finale lors des partiels en janvier (1,5 heure)
- 1 SAÉ - projet en groupe commun à P11 et W11 (lancement en octobre)

Préambule

Qu'attendons-nous de vous ?

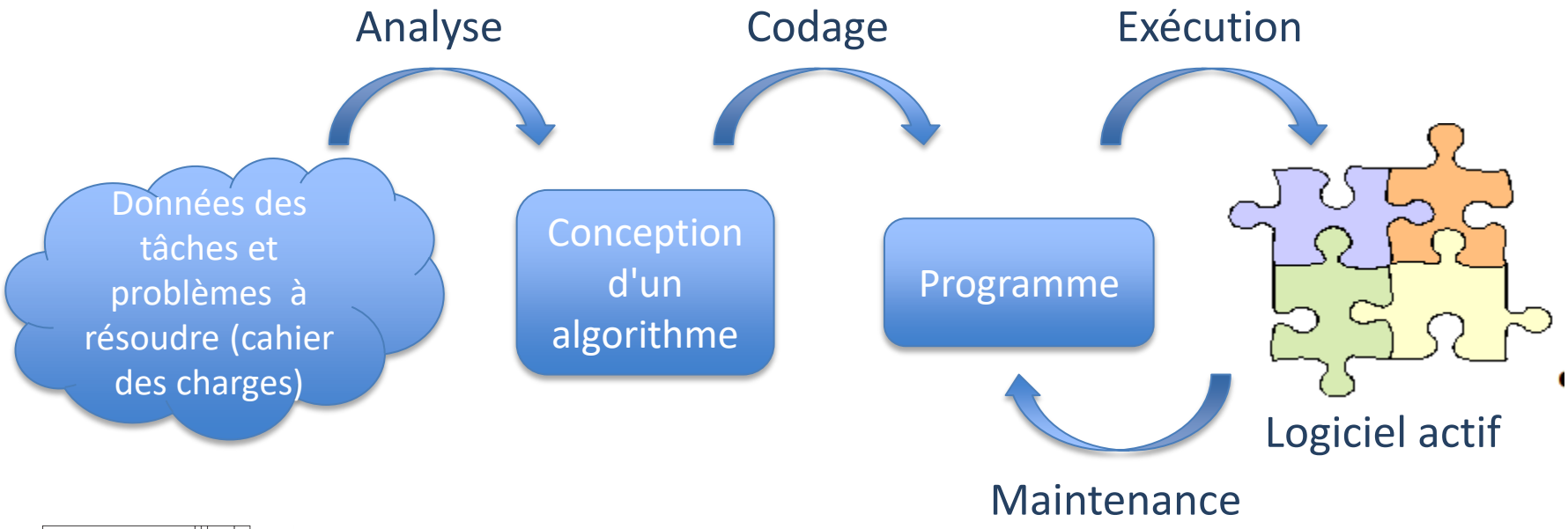
- En général
  - Assiduité,
  - Pas de téléphone !
- En TD (à 25 étudiants)
  - Algorithmes papier (on réfléchit et on résout les problèmes)
- En TP (à 12-13 étudiants)
  - On programme en C# les algorithmes vus en TD (memento algo vs C# fourni),
  - On essaye, on se trompe, on recommence, on persévère…
  - Pas d'IA génératives (Chat GPT, Copilot, Claude, …) !

Plan

1. Introduction

Introduction

- **Programmer** est une activité qui consiste à écrire des instructions qu'un ordinateur pourra exécuter pour accomplir des tâches et résoudre des problèmes. Avant d'écrire ces instructions, il est essentiel de les structurer en concevant un algorithme, une étape de réflexion intellectuelle qui fait abstraction de la future implémentation technique (matérielle et logicielle).



Du problème à l'algorithme

Ecrire un algorithme ou programmer dans un langage, 2 états d'esprit complètement différents :

Algorithme (ou pseudo-code) :

- Je réfléchis au problème
- Je trouve la solution
- J'utilise les concepts génériques

Programmation :

- Je crée le logiciel
- J'utilise les possibilités du langage
- Je m'adapte aux contraintes du langage
- J'optimise l'interface utilisateur

Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?

Qu'est-ce qu'un langage de programmation ?

- L'élément de l'ordinateur qui va exécuter les instructions du programme est le processeur.



Et celui-ci ne comprend que le langage binaire (ou code machine) :

```
11110011 00001111 00011110 11111010
01001000 10000011 11101100 00001000
10111111 00000000 00100000 01000000 00000000
```

Qu'est-ce qu'un langage de programmation ?

- Difficile de coder en binaire… Le langage binaire est traduit de l'assembleur, un langage plus lisible (mnémonique) mais toujours proche du processeur :

```
endbr64
subq $8, %rsp
movl $.LC0, %edi
```

- C'est pour éviter de programmer en assembleur (cf S11 et SAE 13.2) qu'ont été inventés les langages de "haut niveau" : C, C++, Java, C#, JavaScript, Python, PHP, …

Qu'est-ce qu'un langage de programmation ?

- Plusieurs procédés existent pour traduire le code issu d'un langage de programmation en code machine, ce qui permet de classifier les langages de programmation en 3 familles principales :

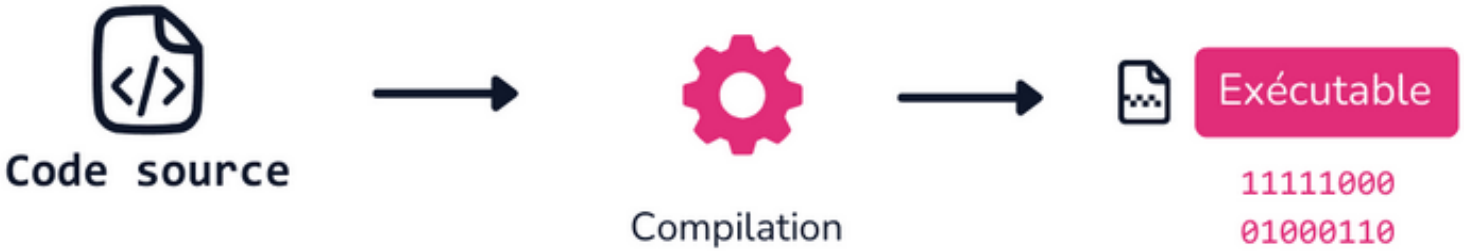
  1 – Langages compilés

  2 – Langages interprétés

  3 – Langage compilés JIT (Just-In-Time)

Qu'est-ce qu'un langage de programmation ?

1 – Les langages compilés (par exemple C, C++)



La compilation consiste à effectuer plusieurs actions :

- Analyse syntaxique,
- Analyse sémantique,
- Optimisation du code,
- Génération du code en assembleur
- Transformation en code machine (assemblage).

Avantage : Exécution très rapide

Inconvénients : Peu portable, nécessite une compilation distincte pour chaque plateforme

Qu'est-ce qu'un langage de programmation ?

2 – Les langages interprétés (par exemple Python)



L'interprétation consiste à effectuer pour chaque ligne plusieurs actions :

- Analyse syntaxique,
- Analyse sémantique,
- Génération directe du code machine

Avantages : Plus portable, c'est l'interpréteur qui est adapté à chaque plateforme.

Inconvénient : Exécution plus lente

Qu'est-ce qu'un langage de programmation ?

3 – Les langages compilés JIT (par exemple Java, C#, Javascript)



La compilation JIT consiste à effectuer pour chaque ligne plusieurs actions :

- Analyse syntaxique,
- Analyse sémantique,
- Génération d'un code intermédiaire bytecode
- Traduction en code machine par la VM (jvm pour Java, mono pour C#, V8 pour Javascript)

Avantages : Code optimisé, très portable, le code intermédiaire peut être exécuté sur les plateformes où la VM existe

Inconvénients : Une couche de complexité supplémentaire, moins rapide qu'un langage compilé

Qu'est-ce qu'un langage de programmation ?

Comparatif des langages selon 3 critères (étude de 2017)

	Energy
(c) C	1.00
(c) Rust	1.03
(c) C++	1.34
(c) Ada	1.70
(v) Java	1.98
(c) Pascal	2.14
(c) Chapel	2.18
(v) Lisp	2.27
(c) Ocaml	2.40
(c) Fortran	2.52
(c) Swift	2.79
(c) Haskell	3.10
(v) C#	3.14
(c) Go	3.23
(i) Dart	3.83
(v) F#	4.13
(i) JavaScript	4.45
(v) Racket	7.91
(i) TypeScript	21.50
(i) Hack	24.02
(i) PHP	29.30
(v) Erlang	42.23
(i) Lua	45.98
(i) Jruby	46.54
(i) Ruby	69.91
(i) Python	75.88
(i) Perl	79.58

	Time
(c) C	1.00
(c) Rust	1.04
(c) C++	1.56
(c) Ada	1.85
(v) Java	1.89
(c) Chapel	2.14
(c) Go	2.83
(c) Pascal	3.02
(c) Ocaml	3.09
(v) C#	3.14
(v) Lisp	3.40
(c) Haskell	3.55
(c) Swift	4.20
(c) Fortran	4.20
(v) F#	6.30
(i) JavaScript	6.52
(i) Dart	6.67
(v) Racket	11.27
(i) Hack	26.99
(i) PHP	27.64
(v) Erlang	36.71
(i) Jruby	43.44
(i) TypeScript	46.20
(i) Ruby	59.34
(i) Perl	65.79
(i) Python	71.90
(i) Lua	82.91

	Mb
(c) Pascal	1.00
(c) Go	1.05
(c) C	1.17
(c) Fortran	1.24
(c) C++	1.34
(c) Ada	1.47
(c) Rust	1.54
(v) Lisp	1.92
(c) Haskell	2.45
(i) PHP	2.57
(c) Swift	2.71
(i) Python	2.80
(c) Ocaml	2.82
(v) C#	2.85
(i) Hack	3.34
(v) Racket	3.52
(i) Ruby	3.97
(c) Chapel	4.00
(v) F#	4.25
(i) JavaScript	4.59
(i) TypeScript	4.69
(v) Java	6.01
(i) Perl	6.62
(i) Lua	6.72
(v) Erlang	7.20
(i) Dart	8.64
(i) Jruby	19.84

Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme

Du problème à l'algorithme

- Définition algorithme :

Selon l'*Encyclopedia Universalis* un algorithme est la

« *spécification d'un schéma de calcul, sous forme d'une suite finie d'opérations élémentaires obéissant à un enchaînement déterminé* ».

- Algorithme vs programme ?

Un algorithme est écrit pour être compréhensible par un humain, alors qu'un programme est écrit de façon à ce qu'il soit compréhensible par une machine.

Du problème à l'algorithme

- Mon problème : Préparer de la pâte à crêpe de blé noir (farine de sarrasin) pour 7 personnes

→Je prends ma recette bretonne ancestrale ;-)



Préparation pour 15 crêpes

Dans un saladier, mettre la farine de blé noir, la farine de froment, le sel, l'œuf entier, l'huile, l'eau et le miel (pour une coloration plus dorée).

Remuer à la fourchette jusqu'à obtenir une boule compacte. Ajouter 10 cl de lait puis battre au fouet jusqu'à obtenir une pâte homogène et sans grumeaux.

Ajouter progressivement le reste de lait. Battre énergiquement la pâte au fouet pour l'aérer et former de petites bulles (c'est le secret pour une pâte moelleuse).

Laisser reposer au frais la pâte minimum 1 heure avant la cuisson.

Du problème à l'algorithme

Analysons cette recette :

De quoi vais-je avoir besoin ?

- Des contenants : saladier, cuillère à café, cuillère à soupe, un verre mesureur
- Des outils : un fouet, un frigo

Un problème qui amène d'autres problèmes…

- J'ai une recette pour 15 crêpes alors que je veux faire des crêpes pour 7 personnes
- Des unités différentes (grammes, cuillère à soupe, unité, litre, cl, cuillère à café)

Du problème à l'algorithme

Un problème qui amène d'autres problèmes…

- J'ai une recette pour 15 crêpes alors que je veux faire des crêpes pour 7 personnes

Quelle information me manque-t-il ?

Du problème à l'algorithme

Un problème qui amène d'autres problèmes…

– J'ai une recette pour 15 crêpes alors que je veux faire des crêpes pour 7 personnes

Quelle information me manque-t-il ?

→**Le nombre de crêpes au blé noir que mange (en moyenne) une personne**

Du problème à l'algorithme

Un problème qui amène d'autres problèmes…

– J'ai une recette pour 15 crêpes alors que je veux faire des crêpes pour 7 personnes

Quelle information me manque-t-il ?

→Le nombre de crêpe au blé noir que mange (en moyenne) une personne

→**Disons 3 crêpes !**

→En conséquence il faut adapter la quantité pour chaque ingrédient

Du problème à l'algorithme

→ En conséquence il faut adapter la quantité pour chaque ingrédient

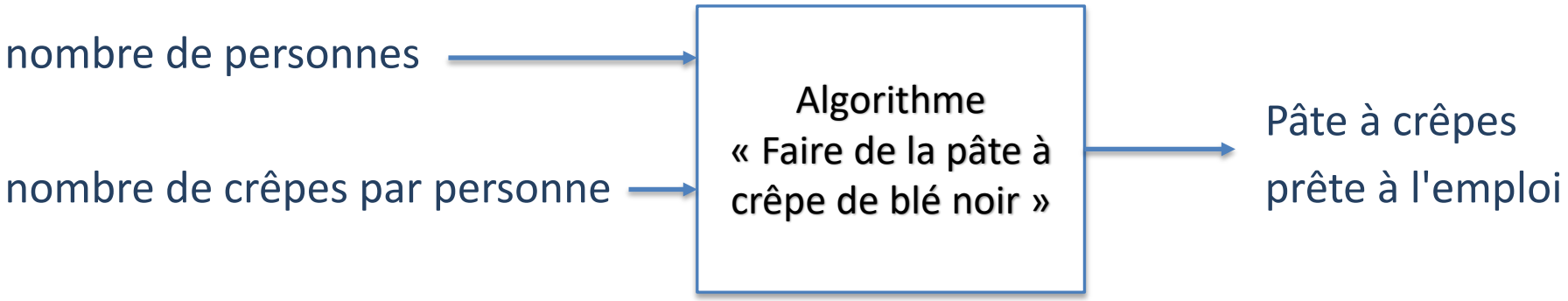
7 personnes x 3 crêpes = 21 crêpes

21 / 15 = 1.4 Donc il faudra majorer les quantités de 1.5 (simplifions)

	15 crêpes	**21 crêpes**
→ Farine de blé noir	500 g	750 g
→ Farine de froment	2 c.s	3 c.s
→ Eau	1 litre	1,5 litre
→ Œuf	1	1,5 œuf (?) → 2 œufs
→ Sel	1 c.s	1,5 c.s
→ Huile	1 c.s	1,5 c.s
→ Lait	25 cl	37,5 cl
→ Miel	1 c.c	1,5 c.c

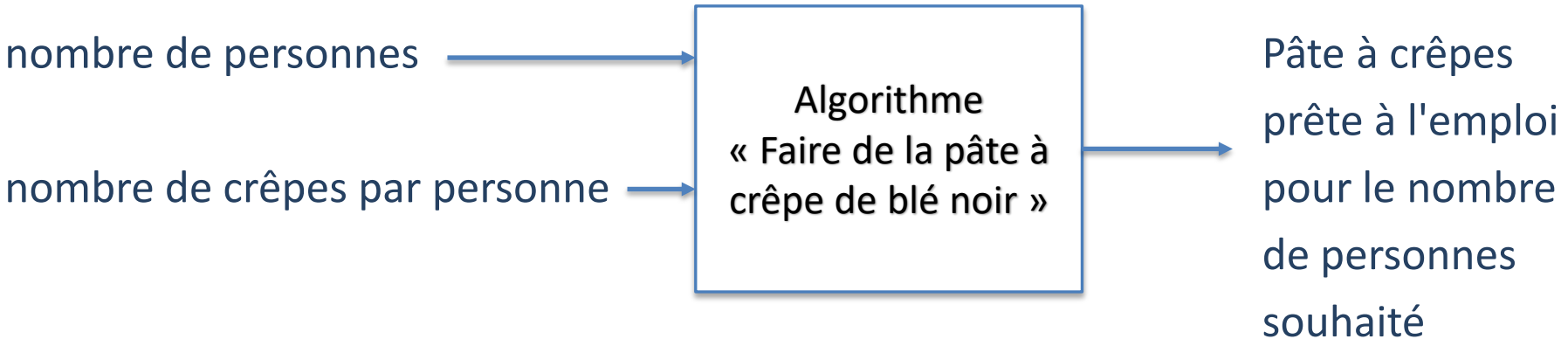
Du problème à l'algorithme

Résumons



Du problème à l'algorithme

Résumons



Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme
4. Exemple d'algorithme

Exemple d'algorithme

```
Algorithme Faire de la pâte à crêpes de blé noir
    Déclarations
        Constantes
            QTE_FARINE_ BN = 500
            QTE_FARINE_F = 2
            NB_OEUF  = 1
            QTE_SEL = 1
            QTE_HUILE = 1
            QTE_EAU = 1
            QTE_LAIT = 25
            QTE_MIEL = 1
            NB_PERS = 15
        Variables
            nbPersonnes
            nbCrepesParPersonne
            totalFarineBN
            totalFarineF
            totalOeuf
            totalSel
            totalHuile
            totalEau
            totalLait
            totalMiel
            ratio
```

Exemple d'algorithme

```
Algorithme Faire de la pâte à crêpes de blé noir
    Déclarations des constantes et des variables
    …
Début
    //Renseigner le nombre de personnes et le nombre de crêpes
    nbPersonne = 7                  // Ces informations pourront
    nbCrepesParPersonne = 3         // être saisies par la suite

    //Calcul du ratio
    ratio = (nbPersonnes * nbCrepesParPersonne) / NB_PERS
    Arrondir  ratio au demi supérieur

    //Calcul des quantités nécessaires
     totalFarineBN  = QTE_FARINE_ BN * ratio
     totalFarineF   = QTE_FARINE_F   * ratio
     totalOeuf   = NB_OEUF * ratio
     Arrondir totalOeuf  à l'entier supérieur
```



Exemple d'algorithme

```
Algorithme Faire de la pâte à crêpes de blé noir
    Déclarations des constantes et des variables
    …
Début
    //Suite …
    totalSel  = QTE_SEL * ratio
    totalHuile  = QTE_HUILE * ratio
    totalEau  = QTE_EAU * ratio
    totalLait  = QTE_LAIT * ratio
    totalMiel  = QTE_MIEL * ratio
```

Exemple d'algorithme

Algorithme Faire de la pâte à crêpes de blé noir

…

Début

… ***//Actions***

Mettre dans un saladier la farine de blé noir, la farine de froment, le sel, l'œuf entier, l'huile, l'eau et le miel

Remuer à la fourchette jusqu'à obtenir une boule compacte

Mesurer à l'aide du verre mesureur 10 cl de lait

Ajouter dans le saladier les 10 cl de lait

Battre au fouet jusqu'à obtenir une pâte homogène et sans grumeaux

Laisser reposer la pâte au frais minimum 1 heure

//La pâte est prête !

Fin

Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme
4. Exemple d'algorithme
5. Les structures algorithmiques

Les structures algorithmiques

Voici "un" algorithme permettant de partir à l'IUT le matin :

```
Algorithme Partir à l'IUT le matin

   Début
1  |  TantQue le réveil ne sonne pas
2  |  |    Dormir
3  |  FinTantQue
4  |  Se lever
5  |  Préparer le café
6  |  Déjeuner
7  |  Se doucher
8  |  S'habiller
9  |  Mettre ses chaussures
10 |  Mettre sa veste
11 |  Si il pleut Alors
12 |  |    prendre le parapluie
13 |  FinSi
14 |  Ouvrir la porte
15 |  Sortir
16 |  Partir pour l'IUT
   Fin
```

Les structures algorithmiques

Observons cet algorithme que chacun déroule de façon intégrée tous les jours :

- l'ordre des actions est important, certaines actions ne peuvent pas être interverties (par exemple les actions 14 et 15), alors que d'autres peuvent l'être sans problème (par exemple les actions 6 et 7)

C'est la **structure séquentielle** qui permet de faire en sorte que les actions s'exécutent les unes après les autres et qu'elles sont disposées dans le bon ordre.

- certaines actions peuvent ne jamais s'exécuter car elles sont dépendantes d'une prise de décision : action 12

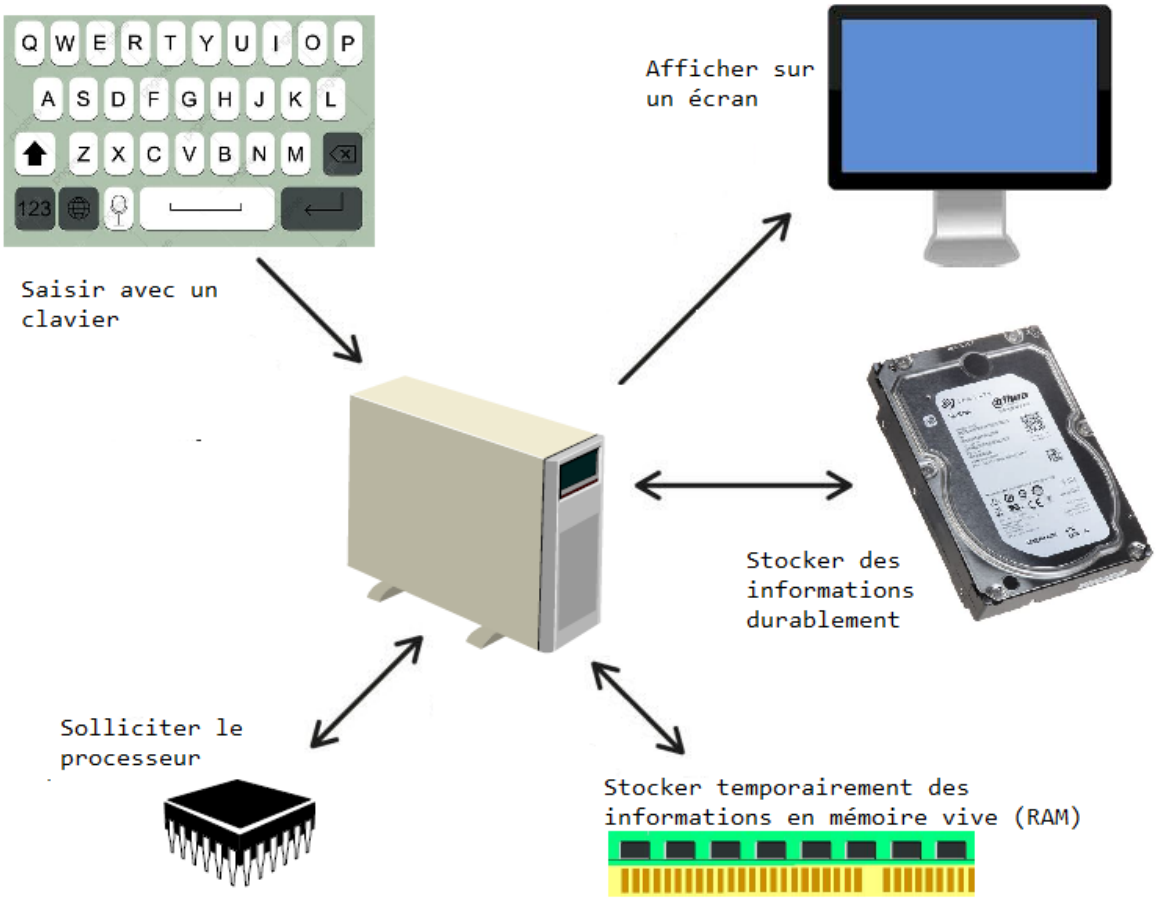
Ce sont les **structures alternatives (ou conditionnelles)** qui permettent de rassembler des actions soumises à une condition.

- certaines actions peuvent s'exécuter plusieurs fois de suite : action 2

Ce sont les **structures itératives (ou répétitives)** qui permettent de rassembler des actions qui seront exécutées plusieurs fois (au sein d'une boucle).

Les structures algorithmiques

- Concernant un algorithme destiné à être codé, voici 5 actions fondamentales qu'un logiciel doit posséder pour piloter l'ordinateur



Les structures algorithmiques

- L'algorithme doit donc être capable de prendre en compte ces 5 actions fondamentales :
  - Stocker temporairement une information en mémoire vive
  - Demander au processeur d'effectuer un calcul
  - Afficher des informations sur un écran
  - Permettre à l'utilisateur de saisir une information
  - Stocker durablement une information

$\rightarrow$ Ces actions seront disposées en respectant la structure séquentielle et au besoin placées au sein des structures alternatives et itératives.

Plan

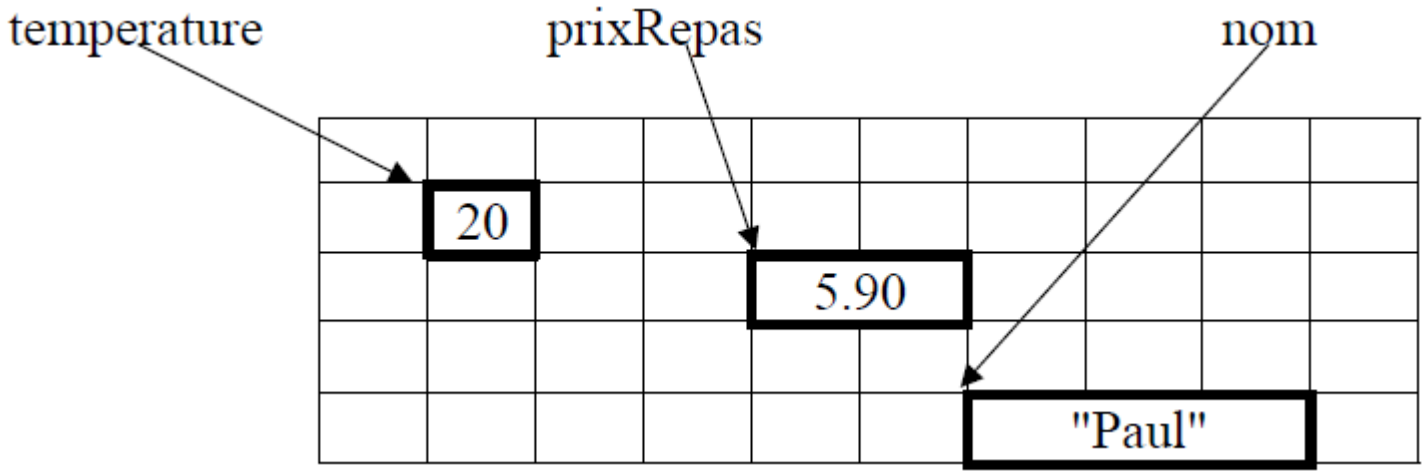
1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme
4. Exemple d'algorithme
5. Les structures algorithmiques
6. La gestion de la mémoire

La gestion de la mémoire

Pour pouvoir être utilisée, une information doit être stockée et facilement atteignable.

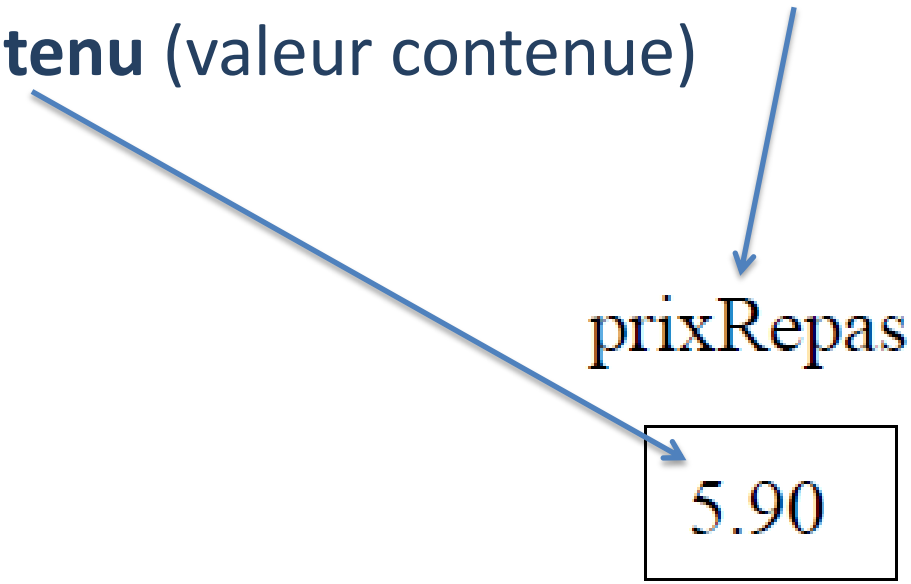
Exemple : Jean fait la connaissance de Paul. Afin de lui dire à la prochaine rencontre "Salut Paul", Jean va devoir stocker l'information "Paul" dans sa mémoire. C'est le fait de voir le visage de "Paul" qui va permettre d'aller rechercher l'information en mémoire (ici le prénom associé). Un ordinateur fonctionne de la même façon !

Chaque information est stockée dans un bloc mémoire auquel on donne un nom.



La gestion de la mémoire

prixRepas est le nom du **contenant** alors que 5.90 est le **contenu** (valeur contenue)



Remarque : aucune allusion ici de l'unité (€), l'algorithme manipule une valeur sans connaître l'unité.

La gestion de la mémoire

Déclaration et utilisation des blocs mémoire dans un algorithme

Lorsqu'on veut stocker une valeur il faut effectuer une déclaration ce qui consiste à :

1. Déterminer à quelle famille ce bloc appartient (**variable** ou **constante**)
2. Nommer ce bloc de façon **explicite** et **conventionnelle**
3. Indiquer le **type** d'information à stocker

La gestion de la mémoire

1) Déterminer à quelle famille ce bloc appartient (**variable** ou **constante**)

Une **constante** est un bloc mémoire dont on connait le contenu dès le début de l'algorithme et qui ne changera pas jusqu'à la fin de l'algorithme.

Une **variable**, au contraire, est un bloc mémoire dont on ne connait pas le contenu dès le début de l'algorithme, de plus ce contenu pourra varier au fil de l'algorithme.

La gestion de la mémoire

2) Nommer ce bloc de façon **explicite** et **conventionnelle**

Voici quelques exemples de nom de variables

Acceptables

temperature
prixRepas
nomClient

Non acceptables

Temperature, température, t° , 1Temperature, a
temperatureMaximaleObserveeDansLaNuit, b
Prix_Repas, prixRepasEn€, prix-repas, c
nomC, nom client, NOMCLIENT, NOM_CLIENT

En déduire les règles de nommage des variables :

-
-
-
-
-

La gestion de la mémoire

2) Nommer ce bloc de façon **explicite** et **conventionnelle**

Voici quelques exemples de nom de variables

Acceptables

temperature
prixRepas
nomClient

Non acceptables

Temperature, température, t° , 1Temperature
temperatureMaximaleObserveeDansLaNuit
Prix_Repas, prixRepasEn€, prix-repas
nomC, nom client, NOMCLIENT, NOM_CLIENT

En déduire les règles de nommage des variables :

- En minuscule sauf 1ère lettre nouveau mot (prix**R**epas)
- Pas de caractères spéciaux, pas d'espace, pas de d'accent
- Pas de chiffre en 1ère position
- Pas de majuscule en 1ère position, ni tout en majuscule
- Nom explicite mais pas trop long

La gestion de la mémoire

2) Nommer ce bloc de façon **explicite** et **conventionnelle**

Voici quelques exemples de nom de constante

Acceptables	**Non acceptables**
TAUX_TVA	tauxTva, Taux_Tva
ANNEE_EN_COURS	annéeEnCours, Annee_En_Cours, A_E_C
MAX	max, Max

En déduire les règles de nommage des constantes :

-
-
-

La gestion de la mémoire

2) Nommer ce bloc de façon **explicite** et **conventionnelle**

Voici quelques exemples de nom de constante

Acceptables	**Non acceptables**
TAUX_TVA	tauxTva, Taux_Tva
ANNEE_EN_COURS	annéeEnCours, Annee_En_Cours, A_E_C
MAX	max, Max

En déduire les règles de nommage des constantes :

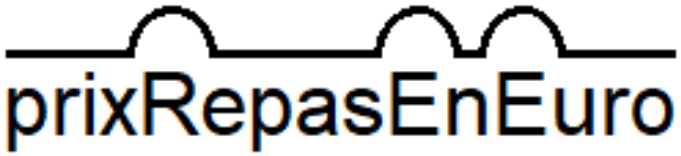
- Tout en majuscule
- un _ (underscore) pour séparer les mots
- nom explicite

Avantages : visuellement vous savez si vous manipulez une variable ou une constante ! Votre code peut être lu par d'autres personnes.

La gestion de la mémoire

Ces deux conventions portent un nom :

- **camelCase** (pour les variables)



- **UPPER_CASE** (pour les constantes)

PRIX_REPAS_EN_EURO = 5.5

Il en existe d'autres en fonction des langages et du type d'objet à nommer :

PascalCase	**PrixRepasEnEuro**
spinal-case (ou **kebab-case**)	**prix-repas-en-euro**
snake_case	**prix_repas_en_euro**

La gestion de la mémoire

3) Déclarer le type d'information à stocker

Lors de la déclaration chaque variable ou constante sera spécialisée en fonction des valeurs qu'elle va stocker. Voici quelques types avec des valeurs possibles :

Types	Exemples
entier	10 597 31621 -23
réel	1.5 1.598740
double	1.5 1.598740100000000
caractère	'a' '&' '9' 'Z'
chaîne de caractères	"IUT Robert Schuman" "Rj 45" "803"
booléen	vrai faux

La gestion de la mémoire

3) Déclarer le type d'information à stocker

Types	Exemples
entier	10 597 31621 -23
réel	1.5 1.598740
double	1.5 1.598740100000000
caractère	'a' '&' '9' 'Z'
chaîne de caractères	"IUT Robert Schuman" "Rj 45" "803"
booléen	vrai faux

- un réel simple a une précision de 6 décimales alors qu'un réel double a une précision de 15 décimales.
- un caractère est entouré d'apostrophes ' ' (ou quotes) alors qu'une chaîne de caractères est entourée de guillemets " "
- Le point '.' est utilisé en informatique pour séparer l'entier et les décimales.

Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme
4. Exemple d'algorithme
5. Les structures algorithmiques
6. La gestion de la mémoire
7. Déclarations dans un algorithme

Déclarations dans un algorithme

- **Mon problème :** Un marchand d'eau minérale livre ses clients strasbourgeois à vélo électrique et voudrait pouvoir afficher un ticket qui récapitule le montant à facturer après avoir saisi le nom du client et la quantité de bouteilles sachant que le prix de la bouteille est de 57 centimes et que la livraison représente 15% de la commande. Voici le ticket que le marchand souhaiterait afficher :

Client : Celler Jacques

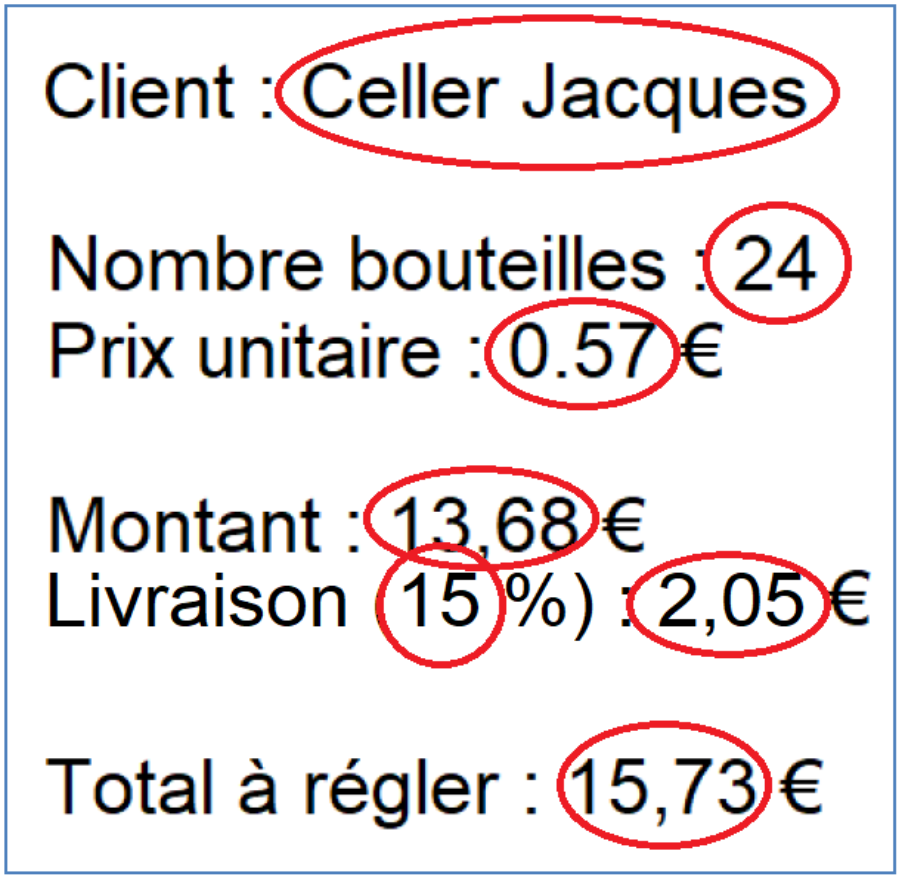
Nombre bouteilles : 24
Prix unitaire : 0.57 €

Montant : 13,68 €
Livraison (15 %) : 2,05 €

Total à régler : 15,73 €

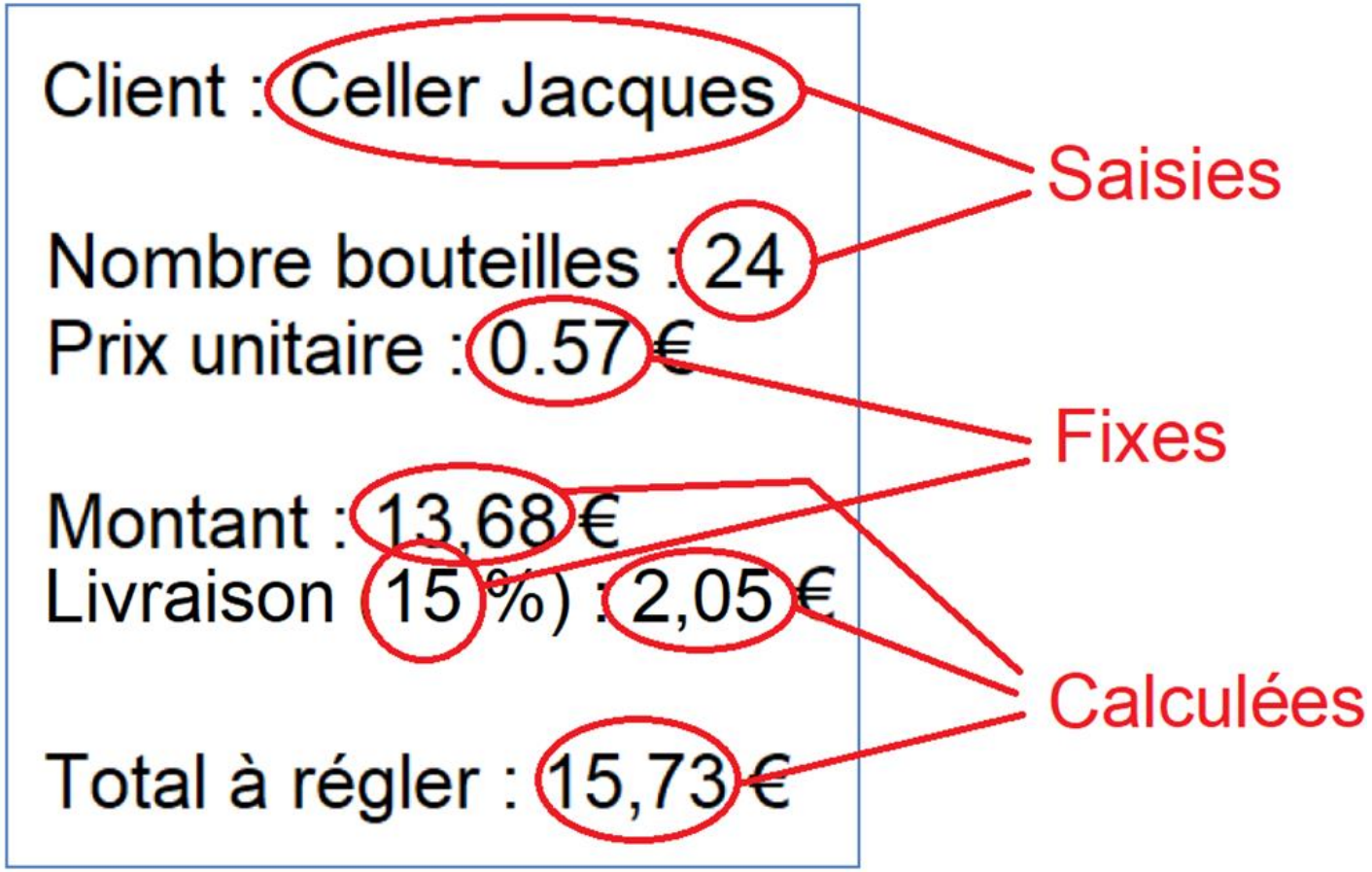
Déclarations dans un algorithme

- **D'où proviennent ces informations ?**



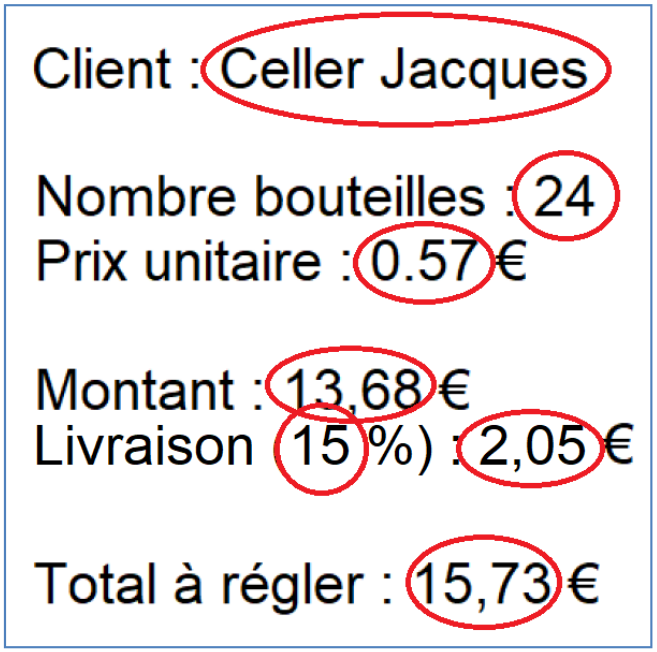
Déclarations dans un algorithme

- **D'où proviennent ces informations ?**



Déclarations dans un algorithme

- **Je vais donc avoir besoin de ces blocs mémoires :**



Variable	Constante	type de donnée	nom
X		chaîne de caractère	nomClient
X		entier	nbBouteilles
	X	réel	PRIX_UNITAIRE
X		réel	montant
X		réel	livraison
	X	réel	TAUX_LIVRAISON
X		réel	total

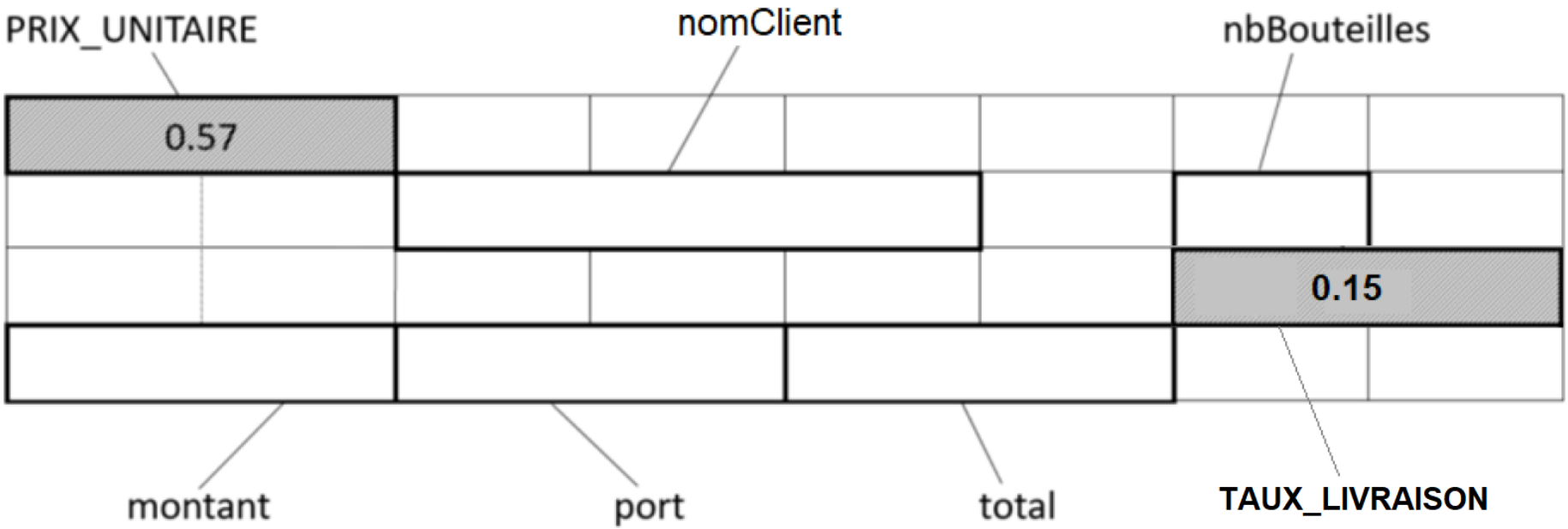
Déclarations dans un algorithme

Suite à l'analyse de la mémoire nécessaire, on peut effectuer la déclaration dans l'algorithme :

```
Algorithme EauxMinerales
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
            TAUX_LIVRAISON = 0.15 : réel
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
            montant, livraison, total : réels
    Début
        …
```

Déclarations dans un algorithme

Voici une représentation possible de la mémoire allouée



Plan

1. Introduction
2. Qu'est-ce qu'un langage de programmation ?
3. Du problème à l'algorithme
4. Exemple d'algorithme
5. Les structures algorithmiques
6. La gestion de la mémoire
7. Déclarations dans un algorithme
8. Utilisation des variables et des constantes

Utilisation des variables et des constantes

1- Je veux affecter une valeur à une constante ou une variable

C'est l'opérateur d'affectation '=' qui permet cette action

Cas d'une constante, l'affectation est effectuée lors de la déclaration

```
Algorithme "EauxMinerales"
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
            TAUX_LIVRAISON = 0.15 : réel
```

Cette valeur ne pourra plus être modifiée dans l'algorithme ! Mais l'an prochain si le prix unitaire passe à 0,59, if suffira de mettre à jour l'algorithme à 1 seul endroit.

Utilisation des variables et des constantes

Cas d'une variable, la ou les affectations sont effectuées dans l'algorithme

```
Algorithme EauxMinerales
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
            montant, livraison, total : réels
 Début
    //Affectation du nombre de bouteilles (initialisation)
    nbBouteilles = 24
    //Affectation suite à un calcul
    montant = nbBouteilles * PRIX_UNITAIRE
```

Utilisation des variables et des constantes

Les opérateurs arithmétiques

+	addition	totalNote = noteA + noteB
-	soustraction	total = prix - remise
*****	multiplication	noteCoef = note * coef
/	division	moyenne = total / nombre
DIV	division euclidienne	moitie = nombre DIV 2
MOD	modulo (reste de la division euclidienne)	reste = nombre MOD 2

Remarque : pour plus de lisibilité, je mets un espace entre chaque opérateur

Utilisation des variables et des constantes

2-Je veux afficher un résultat à l'écran (**action Afficher**)

```
Algorithme DateDuJour
    Déclarations
        Constantes
            MOIS = "septembre" : chaîne de caractères
        Variables
            jour : entier
Début                                                    △ symbolise un espace
    jour = 8
    Afficher "Le" + jour + MOIS + 2026
    Afficher "Le△" + jour + "△" + MOIS + "△2026"
    Afficher "Le△" + jour + "△" + MOIS + "△" + (2024 + 2)
```

Utilisation des variables et des constantes

```
Afficher "Le" + jour + MOIS + 2026
```

Le8septembre2026

△ symbolise un espace

```
Afficher "Le△" + jour + "△" + MOIS + "△2026"
```

Le 8 septembre 2026

```
Afficher "Le△" + jour + "△" + MOIS + "△" + (2023 + 2)
```

Le 8 septembre 2026

Utilisation des variables et des constantes

Pour résumer l'affichage, 3 cas différents se présentent :

- Du texte entre guillemets : par exemple **"Le "** est affiché directement,
- du texte sans guillemet : par exemple **jour** ou **MOIS** font référence respectivement à une variable et une constante, dans ce cas c'est le **contenu** de la variable ou de la constante qui est affiché,
- un nombre sans guillemet : par exemple **2026**, ou directement un calcul entre parenthèses par exemple **(2024 + 2)** est affiché directement après avoir été converti en chaîne de caractères.

Remarque : Ici le **+** est utilisé en tant qu'opérateur de **concaténation** de chaîne, il est sous-entendu qu'une valeur numérique est préalablement convertie en chaîne avant d'être affichée.

Utilisation des variables et des constantes

3-Je veux demander à l'utilisateur de saisir une valeur au clavier et stocker cette valeur (**action Saisir**)

```
Algorithme EauxMinerales
    Déclarations
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
    Début
        //Saisir du nom du client
        Afficher "Saisir le nom du client : "
        Saisir nomClient

        //Saisir le nombre de bouteilles
        Afficher "Quel est le nombre de bouteilles : "
        Saisir nbBouteilles
```

Utilisation des variables et des constantes

```
Algorithme "EauxMinerales"
    …
Début
    //Saisir du nom du client
    Afficher "Saisir le nom du client : "
    Saisir nomClient

    //Saisir le nombre de bouteilles
    Afficher "Quel est le nombre de bouteilles : "
    Saisir nbBouteilles
```

Invite de l'utilisateur

variable à affecter

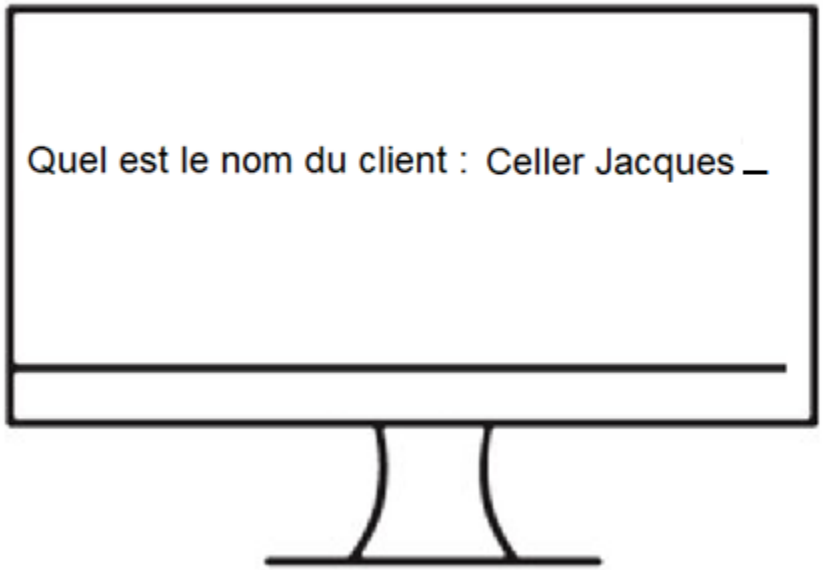
Utilisation des variables et des constantes

Comment ça se passe ?

Action de l'algorithme	Résultat attendu sur l'écran
Afficher "Quel est le nom du client : "	
Saisir nomClient	

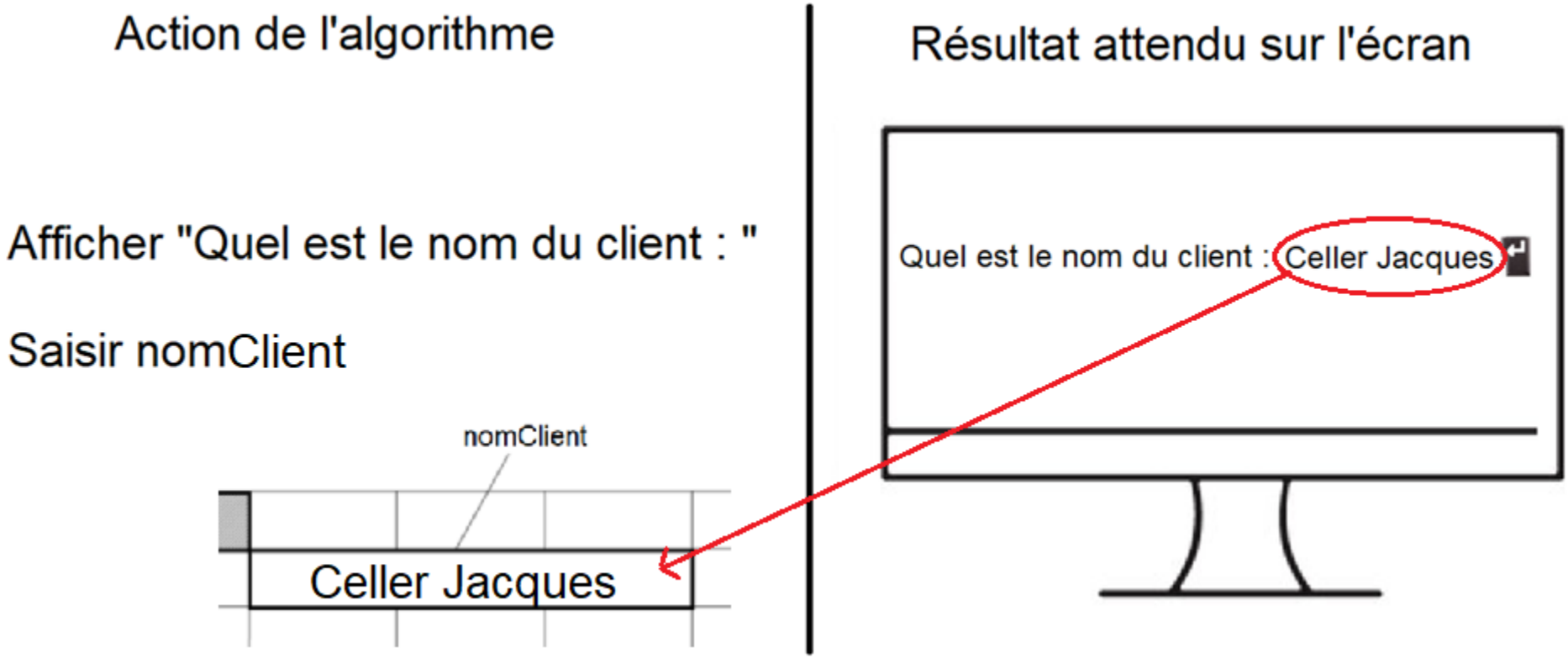
Utilisation des variables et des constantes

Comment ça se passe ?

Action de l'algorithme	Résultat attendu sur l'écran
Afficher "Quel est le nom du client : "	
Saisir nomClient	

Utilisation des variables et des constantes

Comment ça se passe ?



Utilisation des variables et des constantes

On peut maintenant compléter notre algorithme…

```
Algorithme EauxMinerales
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
            TAUX_LIVRAISON = 0.15 : réel
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
            montant, livraison, total : réels
Début
        …
```

Utilisation des variables et des constantes

```
Algorithme EauxMinerales
    …
 Début
    //Saisir le nom du client
    Afficher "Saisir le nom du client : "
    Saisir nomClient

    //Saisir le nombre de bouteilles
    Afficher "Quel est le nombre de bouteilles : "
    Saisir nbBouteilles

    //Calculs
    …
```

Utilisation des variables et des constantes

```
Algorithme EauxMinerales
    …
 Début
    …
    //Calculs
    montant = PRIX_UNITAIRE * nbBouteilles

    livraison = montant * TAUX_LIVRAISON

    total = montant + livraison

    //Affichage du résultat
    …
```

Utilisation des variables et des constantes

```
Algorithme EauxMinerales
    …
Début
    …
    //Affichage du résultat
    Afficher "Client : " + nomClient
    Afficher "Nombre bouteilles : " + nbBouteilles
    Afficher "Prix unitaire : " + PRIX_UNITAIRE + " €"
    Afficher "Montant : " + montant + " €"
    Afficher "Livraison (" + (TAUX_LIVRAISON * 100) + " %) : " +
        livraison + " €"
    Afficher "Total à régler : " + total + " €"
Fin
```

Utilisation des variables et des constantes

Dernière chose… pour être lisible, un algorithme doit être indenté !

```
Algorithme EauxMinerales
    Déclarations
        Constantes
            PRIX_UNITAIRE = 0.57 : réel
            TAUX_LIVRAISON = 0.15 : réel
        Variables
            nomClient : chaîne de caractères
            nbBouteilles : entier
            montant, livraison, total : réels
Début
    //Saisir le nom du client
    Afficher "Saisir le nom du client : "
```

Une indentation est un retrait à droite (touche Tab) qui signifie "**fait partie de**"